

AI support agent on TWCS — SpotifyCares

Every figure below is reproduced by `python scripts/evaluate.py --all` over the artefacts committed in this repository. The LLM stages ran on a local `llama3.2:3b` server; **\$0.00 was spent on any API**. Reproduce with `README.md §3`.

1. Problem framing

Given a customer message sent to a brand on Twitter, the agent must (a) classify intent, (b) decide auto-handle vs escalate with a stated reason, and (c) for auto-handled cases draft a reply grounded in how the brand historically resolved similar issues.

What “good” means here. The expensive error is not a mediocre reply — it is auto-answering something that should have gone to a human: a compromised account, a disputed charge, a message nobody understood. So the headline is the **unsafe-auto rate at matched coverage** (D3): of the items the system chooses to handle automatically, the share a human labelled `ESCALATE`. It is meaningless without coverage, because a system that auto-handles nothing scores 0%.

Intent accuracy is not the headline and is not a proxy for it. They are separate measurements on separate axes: intent accuracy asks whether a label is right, D3 asks what it costs when a label is wrong in the unsafe direction. The two move independently — §5 shows the best intent classifier and the best escalation policy are not the same system.

Brand: SpotifyCares (D1), chosen on measured grounds. Its `MUST`-escalation proxy prevalence is 7.5% against AmericanAir’s 2.5% — 2.9× the density. We gave up a cleaner drafting corpus (Spotify: 24.5% duplicate replies, 97.1% signatures) to get an escalation task with enough positive mass to measure at $n=200$.

Scope boundaries. One brand, English-dominant, first customer message only. TWCS contains **no escalation ground truth**; our labels are a written policy applied by a human annotator (D9). Historical DM-deflection is reported as *behavioural agreement*, never as accuracy — its rate is not even stationary (train 0.369 / dev 0.313 / test 0.431, D16).

2. System design

```
message -> normalise (D10) -> intent -> escalate? + reason
```

```
|
+-- ESCALATE -> deterministic handoff template
|
+-- AUTO_OK  -> retrieve k=3 TRAIN cases
                -> LLM draft
                -> guardrails
                    |
                    +-- pass -> send
                    +-- fail -> handoff (fail-safe)
```

Stage	Implementation	Why
Split	temporal, on thread-opener date (D18)	0 threads straddle a boundary, by construction
Intent	8 classes, 5 attributes, BR-1...BR-11 (codebook 0.2.1)	designed, not discovered: KMeans silhouette ≈ 0 at every k
Escalation	codebook <code>escalation_role</code> $\rightarrow$ decision; 10 ordered text rules $\rightarrow$ reason	policy separated from behaviour (D9)
Retrieval	TF-IDF cosine, 14,491 train openers + their first brand reply	D8 starts at TF-IDF; D15 gates anything heavier
Handoff	11 fixed templates, one per D9 reason	no generation on the escalation path
LLM stages	<code>llama3.2:3b</code> via a local OpenAI-compatible server (D24)	free, offline, reproducible from the committed cache

A draft that trips any guardrail is **discarded and replaced by a handoff template**. Failing safe into an escalation is the only behaviour consistent with the headline metric — though §5 shows it did not buy safety here.

3. Golden set and evaluation methodology

200 items, TEST window only (2017-11-20 → 12-04). Pool: 6,808 openers → 6,495 after removing empty text, pilot items, repeat customers (261), exact duplicates (16) and near-duplicates (4, word-3-gram Jaccard ≥ 0.80).

160 core + 40 targeted. Core is drawn **first**, so it is a genuine simple random sample ($p = 0.0246$) rather than a sample of what the enriched strata left behind. The 40 targeted items cover five boundary pairs the 40-item pilot exposed, seven escalation proxies, and a vague/unclear stratum.

Verified by `sample_golden.py --verify` (13/13): 200 rows, unique ids, all TEST openers, no pilot overlap, one item per thread, 200 distinct customers, text **byte-identical to `twcs.csv`**, no brand reply in the annotation artefact, no stratum leakage, all fields blank at hand-off to the annotator.

Leakage controls are measured, not asserted. `retrieval.py --selftest` over all 200 items: self- and same-thread retrieval structurally impossible (train-only index); **customer-disjointness fired once, on G105**, whose own earlier train opener would otherwise have been retrieved; near-duplicate suppression fired 0 times — it is insurance, not a defeated threat.

Statistics. Wilson 95% on every proportion; paired exact McNemar for every system-vs-baseline comparison (D7). Population statements use core only.

4. Baselines

D23 records the constraint that shapes everything here: **the 200 golden items are the only human labels, and they are TEST**. A conventional supervised baseline therefore cannot be built — fitting on them is the contamination D7 exists to prevent. No supervised metric is reported anywhere in this project.

Baseline	Type	Construction	Sees gold?
rule	trivial	first-match-wins seed regex written on TRAIN before any labelling; nothing fitted	never
distant-lr	simple	TF-IDF + logistic regression fitted on TRAIN openers weakly labelled by those same regexes	never
llm	system	zero-shot llama3.2:3b prompted with the locked codebook; prompts developed on dev	never
lookup-oracle	ceiling, not a baseline	each intent → its majority escalation decision in the golden set	YES

distant-lr is a *distant-supervision proxy*, not a supervised model: its training targets are regex output, so it inherits the seeds' biases.

5. Results

5.1 Intent — the LLM is the worst of the three

Classifier	Core (n=160) the quotable number	Targeted (n=40)	Full (n=200)	macro-F1
rule	39.4% [32.1, 47.1]	80.0%	47.5% [40.7, 54.4]	0.474
distant-lr	42.5% [35.1, 50.2]	80.0%	50.0% [43.1, 56.9]	0.520
llm	38.1% [31.0, 45.8]	57.5%	42.0% [35.4, 48.9]	0.333

The LLM classifier lost. It is 4.4pp below distant-lr on core, 1.3pp below the trivial regex, and its macro-F1 (0.333) is the worst of the three by a wide margin. Paired McNemar says none of these gaps is significant: distant-lr vs llm $b=41$ $c=25$ $p = \mathbf{0.0640}$; llm vs rule $b=27$ $c=38$ $p = \mathbf{0.2145}$; distant-lr vs rule $b=11$ $c=6$ $p = \mathbf{0.3323}$. So the honest statement is that **all three classifiers are statistically indistinguishable at $n=200$, and the point estimates favour the cheapest one**. Neither the logistic

regression nor the language model has earned its place on this evidence — which is exactly the test D15 demands, applied to our own system rather than only to the baselines.

The macro-F1 collapse has a specific cause. llama3.2:3b **never emitted app_device_technical once in 200 predictions**, despite it being the largest gold class (34 items, 17%), and emitted playback_playlist once. F1 = 0.00 on both. 167 of its 200 predictions land in just three classes (other_unclear 73, account_access 53, product_feature_feedback 41). A 3B model given an eight-class codebook used six of the classes in practice.

5.2 Escalation — the D3 headline

Human base rate: **46.9% [39.3, 54.6]** on core (49.5% on all 200, inflated by the enriched strata).

Policy	Coverage	Unsafe-auto	Accuracy	Gold?
always-escalate	0.0%	n/a	49.5%	no
lookup-codebook-rule	25.0%	20.0% [11.2, 33.0]	64.5%	no
lookup-codebook-distant-lr	29.5%	22.0% [13.4, 34.1]	66.0%	no
distant-lr-score	36.5%	21.9% [14.0, 32.7]	70.0%	no
keyword	87.5%	44.0% [36.9, 51.4]	60.0%	no
never-escalate	100.0%	49.5% [42.6, 56.4]	50.5%	no
lookup-oracle	51.0%	6.9% [3.4, 13.5]	93.5%	USES GOLD

Risk-coverage for the only scored policy (distant-lr-score): 5.0% unsafe-auto at 10% coverage, 16.0% at 25%, 24.0% at 50%, 43.9% at 90%. At matched coverage it beats the lookups by only 1.7-4.0pp (p = 0.057 and 0.035).

The precision/coverage trade-off is the whole story. The clean policies look respectable at ~20-22% unsafe-auto only because they auto-handle a quarter to a third of traffic. Pushing coverage to 87.5% (keyword) takes unsafe-auto to 44.0% — barely better than answering everything blind.

5.3 End-to-end, after guardrails — and the guardrails did not help

The table above scores the escalation *decision*. The deployed path adds drafting and guardrails on top, and that changes both numbers:

	Coverage	Unsafe-auto
lookup-codebook-distant-lr decision alone	29.5% (59/200)	22.0% [13.4, 34.1]
after drafting + guardrails (end-to-end)	21.5% (43/200)	27.9% [16.7, 42.7]

Routes: 141 handoff, **16 guardrail fallbacks**, 43 auto-replies. The guardrails discarded 16 of the 59 drafts — and **only one of those 16 was an unsafe auto**. Coverage fell 8pp while the unsafe-auto *rate* rose 5.9pp. On this evidence the guardrail layer is a text-validity filter, not a safety mechanism: it is nearly orthogonal to the thing it fails safe into. That is the honest reading, and it is a stronger argument for keeping it out of the safety story than any number that had flattered it.

5.4 Reason codes and handoff

Every ESCALATE carries exactly one valid D9 reason and every AUTO_OK carries none — **741 escalated and 659 auto-handled** rows across seven policies, 0 missing, 0 illegal, 0 multi-valued, 0 strays. Reason agreement where both human and policy escalate: 52.3% [41.9, 62.6] (n=86) for lookup-codebook-distant-lr, 73.9% for the oracle.

Handoff: **541 rendered replies, 0 guardrail failures, 0 customer-text echoes**, 6-10 distinct replies per policy bounded by 11 templates. render(reason) is a pure lookup — no customer text enters a reply, so a public escalation reply cannot echo back exposed personal data.

5.5 Retrieval

Top-1 cosine median **0.310** (p10 0.219, p90 0.534, min 0.168), which lands on the 0.303 median opener-to-opener similarity measured independently in D13. **This is a weak retriever**, exactly as predicted: a retrieved reply equals the gold reply 0.4% of the time. That is the empirical reason reply quality is judged rather than scored by reference overlap.

5.6 Judge and corruption probes (D11)

llama3.2:3b as judge, 200/200 replies parsed, 0 unparseable: **86.0% acceptable [80.5, 90.1]**. Dimension means: addresses_need 4.74, grounded 4.61, routing 4.24, tone 4.87. §7 explains why 86% is the most misleading number in this report.

Corruption probes — 80 deterministically defective replies built in `judge.py`, no model involved in creating them. Detection = judge marks the reply NOT acceptable:

Defect	Detected	Wilson 95%
missing_escalation	19/20 = 95.0%	[76.4, 99.1]
fabricated_fact	16/20 = 80.0%	[58.4, 91.9]
hostile_tone	14/20 = 70.0%	[48.1, 85.5]
wrong_intent	10/20 = 50.0%	[29.9, 70.1]
pooled	59/80 = 73.75%	

D11 fixed no numeric pass/fail threshold, and none is invented here: choosing one after seeing 73.75% would be post-hoc. The reportable finding is the shape, not a verdict. `wrong_intent` detection is indistinguishable from a coin flip, and wrong intent is precisely the defect §5.1 and §6 show this pipeline produces most often — so the judge is weakest exactly where the system is weakest. That the two other tests of the judge are absent (§8) makes this the only evidence qualifying it.

6. Top 5 failure modes

F1 — Intent collapses, and the LLM did not fix it. Both baselines predict `other_unclear` for 114/200 (rule) and 104/200 (`distant-lr`) against a true 30/200; precision 0.25-0.26, recall 0.90-0.97. Top confusions are one-directional: `app_device_technical` → `other_unclear` (19), `account_access` → `other_unclear` (13-15), `billing_subscription` → `other_unclear` (12-13). Real cases both baselines miss: *G005 “ur app on Xbox is tripping after that update”*, *G013 “server down in LA?”*, *G001 “...make an app for Xbox One but just FYI it really sucks”*. **The stated hypothesis was vocabulary coverage** — a regex cannot generalise “tripping” or “server down” — **and the LLM run was the test of it. The test came back negative.** llama3.2:3b still misses all three (*G001* and *G005* → `product_feature_feedback`, *G013* → `other_unclear`), misses all 34 `app_device_technical` items, and scores *below* the regex on core. **Revised hypothesis:** this is not vocabulary, it is that a 3B model and a regex family both under-use a large eight-class space, and the residual class absorbs whatever they cannot place. A larger model is the obvious next test and was out of budget.

F2 — Classifier failure cascades into over-escalation. Because `other_unclear` carries independent escalation authority, every F1 miss becomes an escalation. `lookup-codebook-rule` escalates **150/200 (75%)** against a 49.5% human rate. *G001*, *G004* (“dying to have the entire reputation album”), *G005* are all gold `AUTO_OK` and all escalated. **Hypothesis:** the safety default is correct in isolation but amplifies upstream error — a classifier problem wearing a policy costume.

F3 — Reason codes collapse to ambiguous. Only 24.8% of reasons come from a text rule; 106 of 141 escalations fall through to the intent-derived fallback, and since the predicted intent is mostly `other_unclear` the fallback is ambiguous. Human `payment_dispute` → predicted ambiguous 11×, including *G014 “trying to pay ... with PayPal and it’s not going through”*. **Hypothesis:** same root cause as F1, measured a second time downstream.

F4 — The taxonomy has no class for rights/creator disputes. Two gold items carry `intent=legal_rights`, which is an escalation *reason*, not one of the eight intents: *G158* (GDPR / “no Disconnect from Facebook button”) and *G179* (a copyright complaint about a re-posted track). No classifier can emit that value, so all three count them as errors; accuracy excluding them is reported alongside (llm 42.4%, `distant-lr` 50.5%, rule 48.0%). `cannot_represent` — the documented escape hatch — was used **zero times** in 200 items. **Left as annotated.** §7 explains why it matters.

F5 — A hard text-only ceiling. At least three items cannot be resolved from text at all: *G047*, *G081* (“sort your app out!” + link), *G191* (“I WISH IT WAS THIS SIMPLE” + link). The annotator marked all three low-confidence with notes saying the linked image is required. **Hypothesis:** these are not model errors and should be reported as an irreducible ceiling, not counted against the system.

7. What is misleading about my headline number?

The 86.0% judge-acceptable rate is the worst offender, and it is ours. Of the 200 replies the judge scored, **157 are deterministic templates** — 141 handoffs drawn from 10 fixed strings and 16 guardrail fallbacks that are all *one* string. Only **43 are generated text**. There are 53 distinct reply strings among 200 judged rows, so the effective sample is roughly 54, not 200. Broken out: handoff_fallback 16/16 = **100%** (one sentence, judged sixteen times), handoff 120/141 = 85.1%, and the part that actually tests the generator, auto_reply, **36/43 = 83.7%**. The headline mostly measures eleven strings a human wrote by hand.

The judge is the same model as the generator. D20 anticipated same-family bias and proposed a tier gap as mitigation. In the run that shipped there is no gap at all: llama3.2:3b drafted the replies and llama3.2:3b judged them. Its 86% approval of its own output should be read with that in front of it, and it is the single strongest reason the corruption probes — which a self-favouring judge cannot pass by flattery — carry the D11 weight rather than the acceptance rate.

The 80.0% targeted intent accuracy is circular. Those targeted:<intent> items were *selected because a seed family matched them*; rule is those same seeds and distant-lr is fitted on their output. The honest number is the core column. The new run gives this a clean check: the LLM, which is **not** derived from the seeds, scores 57.5% on the same stratum against 80.0% for both seed-derived classifiers, and its targeted-to-core gap is 19pp against their 38-41pp. The circularity is now measured rather than merely argued.

Four more ways these numbers could mislead:

1. **Unsafe-auto without coverage is meaningless.** always-escalate scores a perfect 0% and is useless. Every comparison is at matched coverage, and the end-to-end figure in §5.3 (21.5% coverage) is the one that describes the deployed system.
2. **lookup-oracle is not a result.** It uses gold intent *and* gold escalation. It reproduces 93.5% of human decisions, which means **escalation is very nearly a function of intent**, so an escalation model that has merely learned the intent will read as a safety result. That is why the codebook lookup is a mandatory baseline.
3. **No threshold was tuned.** The 0.5 cut is a priori, because no labelled dev set exists. A tuned operating point would score better and could not be defended.
4. **Two unresolved rows move a falsification criterion.** other_unclear is at **exactly 15.0%** (core and full). The taxonomy's own criterion is ">15% = inadequate". If G158/G179 resolve to other_unclear per the codebook, it becomes 15.6% core / 16.0% full — **over the line**. The verdict on the locked taxonomy turns on two rows, and this is reported rather than resolved.

8. Limitations and risks

- **D11's human-agreement half was not completed, and no agreement figure is claimed.** A 60-row rating sheet exists at data/golden/human_ratings_lookup-codebook-distant-lr.csv, but its contents were **not produced by a human annotator** — they were AI-generated review suggestions, and every row was marked as such when written. They are retained unaltered as an artefact of the attempt and must not be read as human ratings. No judge-human κ is reported anywhere in this report. D11 is therefore validated by its corruption probes only, which D11 itself calls the sharper of the two tests — but half of D11 is missing and that is a gap, not a design choice.
- **Three design faults found in the rating instrument**, reported because they would have to be fixed before any future human pass is meaningful: grounded asks whether a reply matches historical evidence, but build_judge_input() passes only the customer message and the reply — **neither the judge nor a human rater ever sees the retrieved evidence**; the generated sheet carries no rubric text or scale anchors, while the judge gets both; and the judge has a hard override (fabrication / missed escalation / hostile tone = not acceptable) that is never conveyed to the rater.
- **One annotator, one pass, and no reliability ceiling exists at submission.** The D12 retest (30 blind items, re-IDed, seed 1212) is drawn and verified 9/9, with a **pre-registered earliest start of 2026-09-19** — two days after this deadline. The gate was set on 09-14, before the collision mattered, and it was **not moved to fit the deadline**. Consequently no agreement number in this project has the denominator that would make it interpretable, and that is the price of honouring the pre-registration. retest_sample.py --verify prints the gate as [WAIT]; see docs/retest.md §5.
- **Reason-vocabulary gaps, reported not patched.** public_pii: the human used it twice (G126, G154) and the PII rule fired on neither. ambiguous: used 15× by the human, reachable only via

fallback, never by evidence. Both were found *after* the rules were frozen; adjusting them now would be tuning on test.

- **Unmeasurable reason codes.** `safety_abuse = 0` (4 eligible items in a 6,495 pool), `high_frustration = 1`, `public_pii = 2`. Report as *not measurable*, never as zero error.
- **Escalation power is thin.** 9 MUST-proxy items in the unbiased core.
- **Not a representative sample.** 40/200 come from enriched strata with inclusion probabilities up to 68× the core rate; the pool is restricted to threads that received a brand reply; one-per-thread/customer removes 261 repeat-customer openers.
- **Prevalence projections were wrong.** `playback_playlist` was projected at 20–30% and measured 8.8% [5.3, 14.2]; `billing_subscription` 6–9% → 16.9%. Corrected in `docs/taxonomy.md` §2.1.

9. What one more week would change

1. **Complete D11's human half.** 60–80 genuine ratings, after fixing the three instrument faults above — above all, showing the rater the same retrieved evidence the grounded dimension asks about. This is the largest single gap in the submission.
2. **Complete the D12 retest** on 09-19, giving every agreement figure the ceiling it currently lacks. The artefact is already drawn; this costs one labelling session and no compute.
3. **Re-run the LLM stages on a larger model.** F1's revised hypothesis predicts the eight-class collapse is capacity-limited, and a 3B model that never emits the largest class is a cheap thing to falsify. Judge and generator should come from different model families this time, which would also retire D20.
4. **Rule on G158/G179**, which settles the taxonomy-adequacy question.
5. **Build a labelled dev set** — the single highest-value change. It would unlock a real supervised baseline, tuned thresholds, and a defensible operating point, all three of which D23 currently forbids.
6. **Only then** revisit retrieval. A 0.310 median cosine is weak, but D15's bar is an ablation showing embeddings beat TF-IDF on *this* task, and without a dev set that ablation cannot be run honestly.